

The University of Azad Jammu & Kashmir, Muzaffarabad



Open Ended Lab

Name: Syeda Khadija Hassan

Roll no: 2024-SE-09

Course Title: CA&LD

Submitted to: Engr. Sidra Rafique

Submission Date: 5th Sep, 2025

Department of Software Engineering

Lab Objectives:

1. Digital Logic Design with PLAs: To design, implement, and verify a 4-bit even parity generator and checker circuit by configuring a Programmable Logic Array (PLA) within a digital logic simulation environment.

2. ALU Operation Emulation: To investigate fundamental Arithmetic Logic Unit (ALU) operations through emulation by writing and executing a low-level program that demonstrates the functionality of shift instructions (SHL, SHR, SAR, SAL).

Task 01

Even Parity Bit Generator & Checker

1. Background Knowledge (Parity Method)

1.1. Concept of Parity for Error Detection

Parity is a simple and widely used technique for **error detection** in digital data transmission and storage systems. It is designed to detect single-bit errors, which are the most common type of error. The core principle involves adding an extra bit, called a **parity bit**, to a string of binary data. The value of this bit is chosen to make the total number of 1s in the entire word (including the parity bit) either even or odd.

1.2. Even vs. Odd Parity

- **Even Parity:** The parity bit is set so that the total number of 1s in the data word plus the parity bit is **even**.
- **Odd Parity:** The parity bit is set so that the total number of 1s is **odd**.

1.3. Generator and Checker Circuits

- A **Parity Generator** is a circuit that creates this parity bit at the transmitting end.
- A **Parity Checker** is a circuit at the receiving end that examines the incoming data word (including the parity bit). If the number of 1s has the correct parity (e.g., even), it is assumed **no error** has occurred. If the parity is incorrect, it signifies that a **single-bit error** has occurred.

1.4. Limitations of the Method

It is crucial to note that the parity method can only detect an **odd number of bit errors**. If an even number of bits flip, the parity will remain correct, and the error will go undetected. This simplicity makes it fast and efficient for environments where single-bit errors are the primary concern, but it is not suitable for environments with high error rates.

1.5. Scope of This Lab

For this lab, we will implement an **even parity** system for a 4-bit data word, including both the generator and checker circuits.

2. Equipment / Tools Used

The following software tool was used for the design, simulation, and implementation of the **Even Parity Generator and Checker**:

2.1. Electronic Workbench: This industry-standard electronic schematic capture and simulation software was used to design and simulate the digital logic circuits, including the Programmable Logic Array (PLA) implementation of the parity generator and checker. Its integrated digital logic simulator allowed for thorough testing and validation of circuit functionality

3. Truth Table for 4-Bit Even Parity Generator

Inputs: 4 Data Bits

Output: Parity Bit (P) - Set to make the total number of 1s (in A,B,C,D,P) even.

Boolean Equation: $P = A \oplus B \oplus C \oplus D$

A	B	C	D	P (Even Parity Bit)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0

0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Truth Table for 5-Bit Even Parity Checker

Inputs: 4 Received Data Bits + 1 Received Parity Bit (P_r)

Output: Error Flag (E) - **E=1** if error is detected (odd number of 1s), **E=0** if no error (even number of 1s).

Boolean Equation: $E = A \oplus B \oplus C \oplus D \oplus P_r$

A	B	C	D	P _r	E (Error Flag)	Interpretation
0	0	0	0	0	0	No Error ✓
0	0	0	0	1	1	Error ✗
0	0	0	1	0	1	Error ✗
0	0	0	1	1	0	No Error ✓
0	0	1	0	0	1	Error ✗
0	0	1	0	1	0	No Error ✓
0	0	1	1	0	0	No Error ✓
0	0	1	1	1	1	Error ✗
0	1	0	0	0	1	Error ✗
0	1	0	0	1	0	No Error ✓
0	1	0	1	0	0	No Error ✓
0	1	0	1	1	1	Error ✗
0	1	1	0	0	0	No Error ✓
0	1	1	0	1	1	Error ✗
0	1	1	1	0	1	Error ✗

0	1	1	1	1	0	No Error ✓
1	0	0	0	0	1	Error ✗
1	0	0	0	1	0	No Error ✓
1	0	0	1	0	0	No Error ✓
1	0	0	1	1	1	Error ✗
1	0	1	0	0	0	No Error ✓
1	0	1	0	1	1	Error ✗
1	0	1	1	0	1	Error ✗
1	0	1	1	1	0	No Error ✓
1	1	0	0	0	0	No Error ✓
1	1	0	0	1	1	Error ✗
1	1	0	1	0	1	Error ✗
1	1	0	1	1	0	No Error ✓
1	1	1	0	0	1	Error ✗
1	1	1	0	1	0	No Error ✓
1	1	1	1	0	0	No Error ✓
1	1	1	1	1	1	Error ✗

Implementation in EWB

Circuit Synthesis for Even Parity Generator

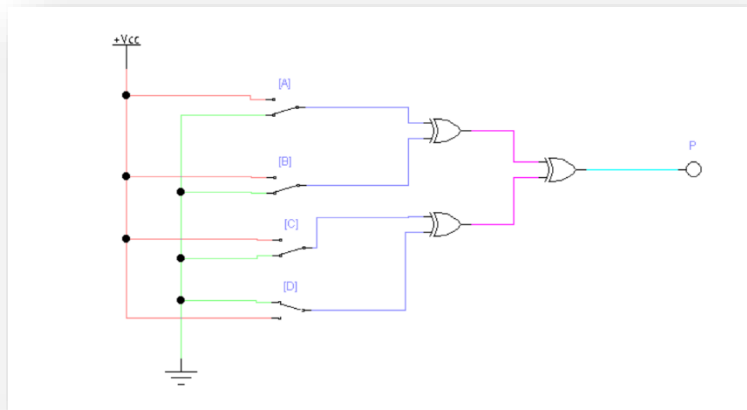
The simplified equation $P = A \oplus B \oplus C \oplus D$ was directly translated into a digital circuit. The implementation utilizes a cascade of three XOR gates:

- The first XOR gate computes the parity of bits A and B ($A \oplus B$).
- The second XOR gate takes this result and computes the parity with bit C ($(A \oplus B) \oplus C$).
- The final XOR gate takes that result and computes the parity with bit D, yielding the final output $((A \oplus B) \oplus C) \oplus D = P$.

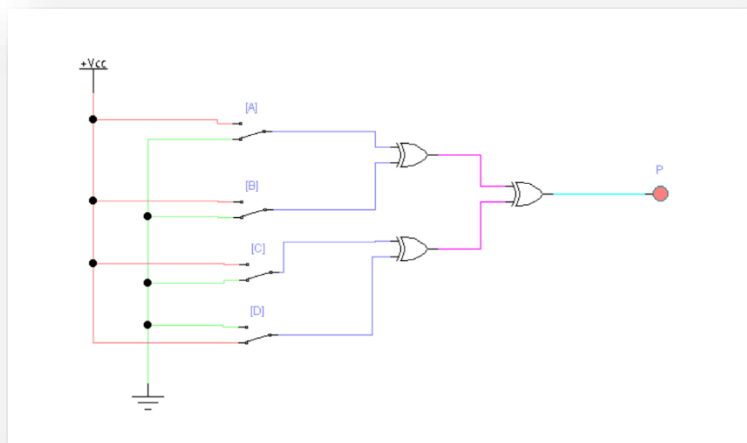
This cascaded structure is efficient, minimal, and leverages the fundamental property of the XOR gate: its output is '1' if and only if an odd number of its inputs are '1'.

Testing

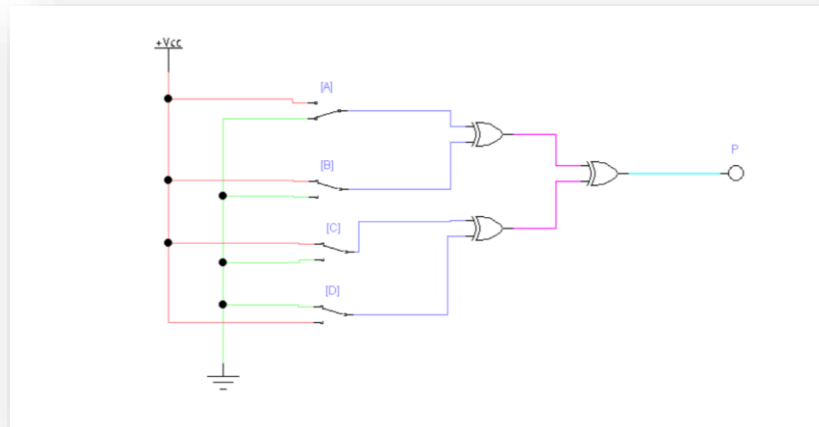
Case 1: All Zeros Case



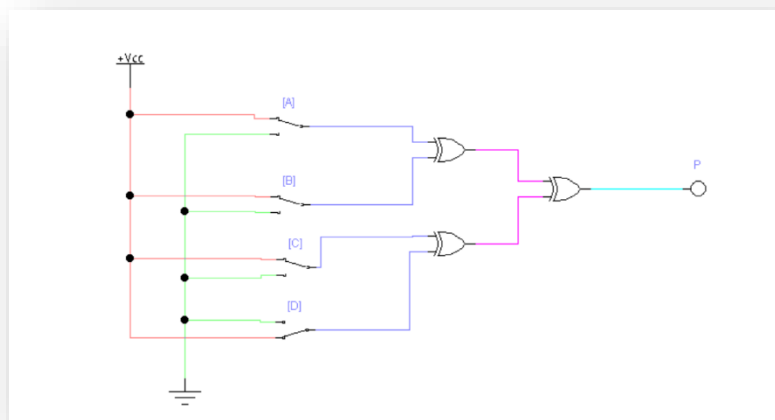
Case 2: The single '1' Odd



Case 3: Even Number of 1s



Case 4: All-Ones



Selected Test Cases and Results

#	Input (A B C D)	Expected P	Observed P	Result
1	0 0 0 0	0	0	✓ PASS
2	0 0 0 1	1	1	✓ PASS
3	0 1 1 0	0	0	✓ PASS
4	1 1 1 1	0	0	✓ PASS

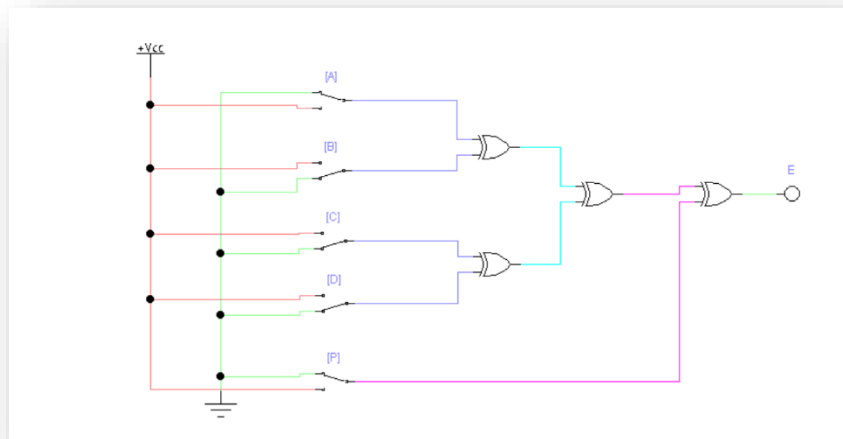
Circuit Synthesis for Even Parity Checker

The simplified equation $E = A \oplus B \oplus C \oplus D \oplus P$ was directly translated into a digital circuit. The implementation utilizes a cascade of four XOR gates:

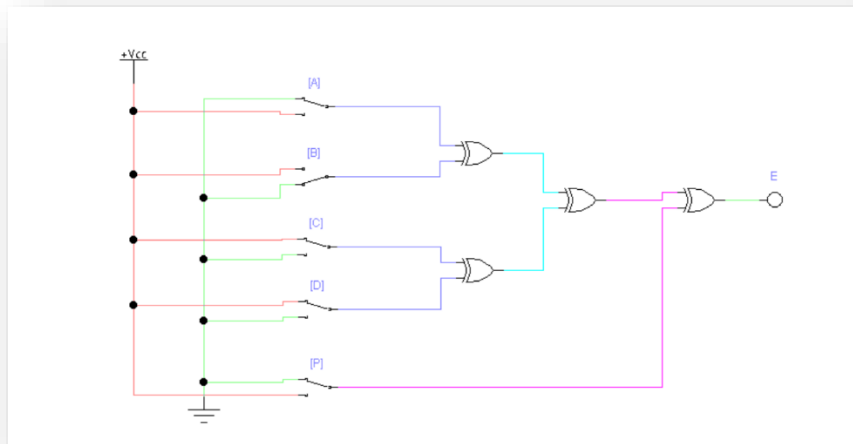
- The first XOR gate computes $A \oplus B$.
- The second takes that result and computes $(A \oplus B) \oplus C$.
- The third takes that result and computes $(A \oplus B \oplus C) \oplus D$.
- The fourth and final XOR gate takes the result of all four data bits and computes the overall parity by adding the received parity bit: $(A \oplus B \oplus C \oplus D) \oplus P$.

Testing

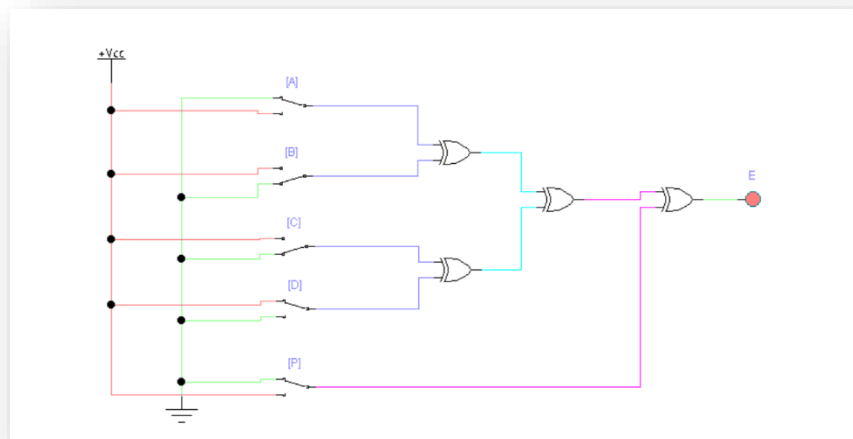
Case 1: All Zeros



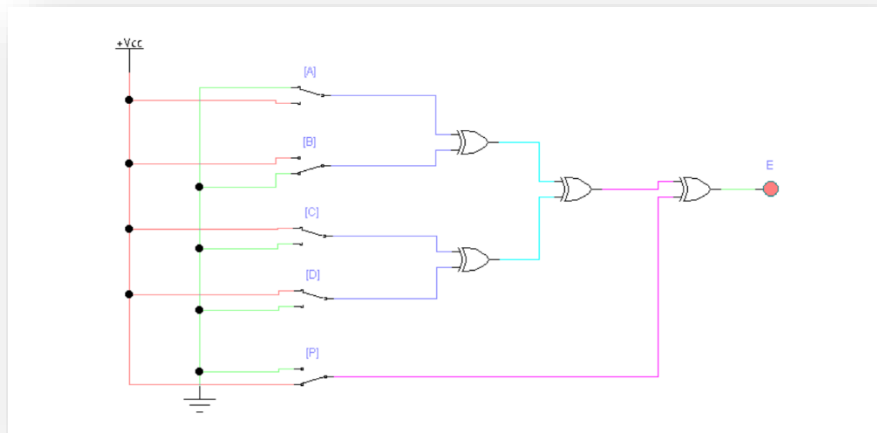
Case 2: Valid data with 1s



Case 3: Single bit Error in data



Case 4: Error in the Parity bit itself



Selected Test Cases and Results for Parity Checker

#	Scenario	Input (A B C D P_r)	Expected E	Observed E
1	No Error	0 0 0 0 0	0	0
2	No Error	0 0 1 1 0	0	0
3	Error (Data Bit)	0 0 0 1 0	1	1
4	Error (Parity Bit)	0 0 1 1 1	1	1

Task 2

Shift Instructions using an Emulator

Background Knowledge: Shift Instructions

Shift instructions are core ALU operations that move the bits in a binary number left or right. They are used for fast arithmetic and bit manipulation.

• SHL (Shift Left) / SAL (Shift Arithmetic Left):

- **Action:** Shifts all bits left. The MSB moves to the Carry Flag (CF), and a 0 is inserted in the LSB.
- **Effect:** Equivalent to **multiplying** by 2 per shift. SHL and SAL are identical.

• SHR (Shift Logical Right):

- **Action:** Shifts all bits right. The LSB moves to the CF, and a 0 is inserted in the MSB.
- **Effect:** Treats the number as **unsigned**. Equivalent to **dividing** by 2 per shift.

• SAR (Shift Arithmetic Right):

- **Action:** Shifts bits right while **preserving the sign bit** (MSB). The LSB moves to the CF, and the MSB is copied.
- **Effect:** Treats the number as **signed** (two's complement). Equivalent to **signed integer division** by 2 per shift.

Application: These instructions provide a faster alternative to multiplication and division by powers of two and are essential for low-level bit-wise operations.

Tools and Environment

- **Emulator Used:** emu8086
- **Architecture:** x86 (16-bit)
- **Language:** x86 Assembly Language

Program Code with Explanation

The following program initializes registers with specific values and performs each shift operation, demonstrating their unique effects on data and flags.

```

mov al, val
mov dx, offset msg1
call PrintString
mov ah, 2
mov dl, al
call PrintHex8Bit      ; print AL in hex

; SHL (Logical shift left by 1)
mov bl, al
shl bl, 1

mov dx, offset msg2
call PrintString
mov ah, 2
mov dl, bl
call PrintHex8Bit

; SHR (Logical shift right by 1)
mov bl, al
shr bl, 1

mov dx, offset msg3
call PrintString
mov ah, 2
mov dl, bl
call PrintHex8Bit

; SAR (Arithmetic shift right by 1)
mov bl, al
sar bl, 1

mov dx, offset msg4
call PrintString
mov ah, 2
mov dl, bl
call PrintHex8Bit

; SAL (Arithmetic shift left by 1, same as SHL)
mov bl, al
sal bl, 1

mov dx, offset msg5
call PrintString
mov ah, 2
mov dl, bl
call PrintHex8Bit

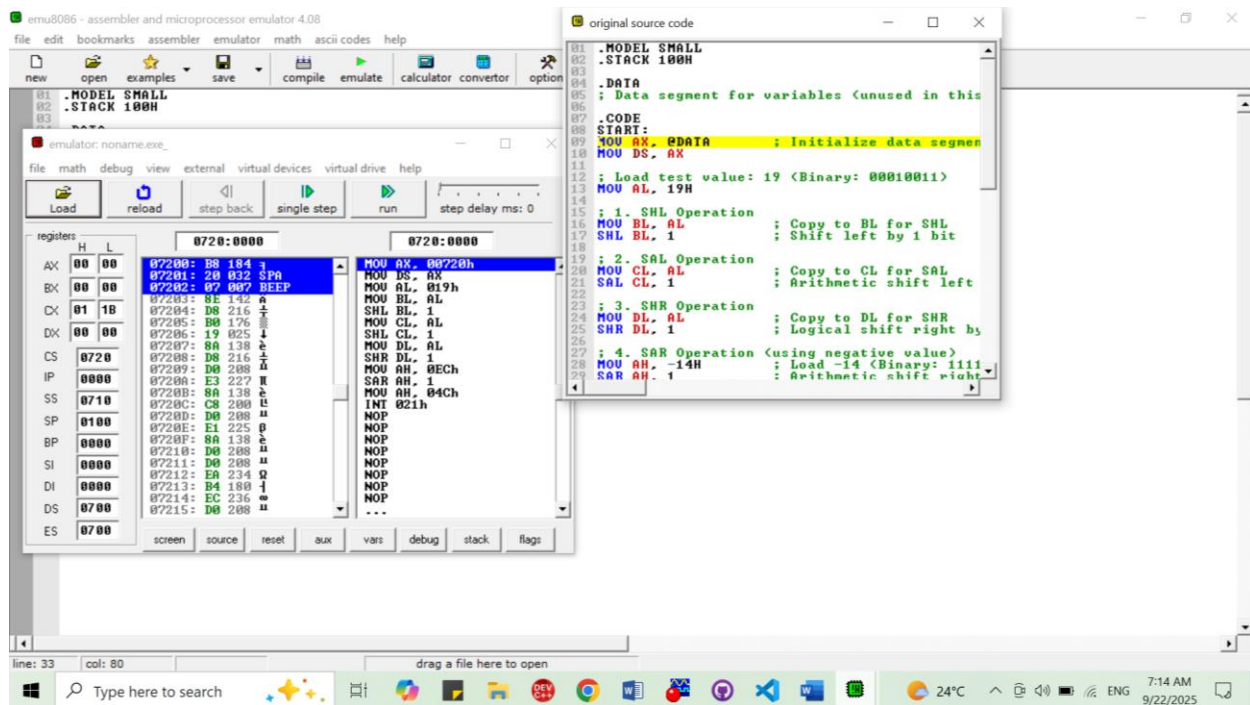
```

Code Explanation:

- The program uses the **AL** and **AH** 8-bit registers to hold initial values.
- **SHL** and **SAL** are demonstrated on the positive value 19.
- **SHR** is demonstrated on the positive value 19.
- **SAR** is demonstrated on the negative value -14 to show sign preservation.
- Each instruction shifts the value by **1 bit position**.

Testing and Results

The program was executed in the emu8086 emulator. The **Step-Into** feature was used to execute one instruction at a time, allowing the observation of register and flag changes after each operation.



Conclusion:

- The **parity generator** ensures even parity by XORing all input bits.
- The **parity checker** validates received data by checking parity consistency.
- The **assembly language program** demonstrates the functionality of shift instructions, helping differentiate arithmetic and logical shifting.